VERIQTA

Learn
Practice
Operate
Grow

Red Hat® Enterprise Linux

ADMINISTRATION HANDBOOK

From Linux Administration Fundamentals to Production RHEL Operations

MANAGE SYSTEMS | CONFIGURE SERVICES | SECURE INFRASTRUCTURE | MANAGE STORAGE | TROUBLESHOOT AND RECOVER

- PRACTICAL EXAMPLES
- REAL-WORLD SCENARIOS
- PRODUCTION BEST PRACTICES
- TROUBLESHOOTING GUIDES
- STEP-BY-STEP LEARNING
- JUNIOR TO PROFESSIONAL

Linux for Real Work

BUILD. ADMINISTER. SECURE. TROUBLESHOOT. KEEP THINGS RUNNING.

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

RHEL EXPLAINED,

WHAT YOU ARE ACTUALLY ADMINISTERING

RED HAT ENTERPRISE LINUX

Red Hat

A stable. secure. enterprise-grade Linux operating system.

1 WHAT IS RED HAT ENTERPRISE LINUX?

Red Hat Enterprise Linux (RHEL) is a commercial, enterprise-grade Linux distribution developed and supported by Red Hat.

- Built for stability, security and long-term use
- Used in production environments
- Comes with official support, updates and certifications
- Widely used in enterprises, cloud and data centers

2 LINUX DISTRIBUTION vs LINUX KERNEL

LINUX KERNEL

- The core of the operating system
- Manages hardware
- Handles processes, memory, devices, etc.
- Developed by the open source community

LINUX DISTRIBUTION

- A complete operating system built around the kernel
- Includes tools, system utilities, package manager and configuration
- RHEL is a Linux distribution that uses the Linux kernel

3 WHY ENTERPRISES USE RHEL

- Stability and reliability
- Long-term support (LTS)
- Regular security updates
- Certified for enterprise hardware and software
- Strong support from Red Hat
- Widely used in cloud, data centers and on-premises
- Trusted by governments, financial institutions and global companies

4 RHEL LIFECYCLE AND ENTERPRISE STABILITY

RHEL MAJOR VERSION (e.g. RHEL 9)

Release	Full Support	Maintenance Support	End of Life (EOL)
New major version	Security updates and bug fixes	Continued security updates	No more updates

Each RHEL major version is supported for many years (typically 10+ years), giving enterprises stability and predictability.

5 RHEL vs FEDORA vs CENTOS STREAM

Feature	RHEL	Fedora	CentOS Stream
Purpose	Enterprise production	Community development	Upstream to RHEL
Stability	Very stable	Latest features (less stable)	More stable than Fedora
Support	Commercial support	Community support	Community support
Update cycle	Long-term (LTS)	Frequent	Rolling updates
Best for	Production environments	Testing and learning	Development and early testing

6 WHERE RHEL APPEARS IN DEVOPS AND INFRASTRUCTURE

CLOUD	AWS, Azure, GCP (RHEL images)
ON-PREMISES	Enterprise data centers
CONTAINERS	RHEL as container base images
CI/CD	Build and deployment environments
KUBERNETES	Worker nodes and container hosts
INFRASTRUCTURE	Web, database, monitoring, and more

7 THE RHEL MENTAL MODEL

HARDWARE → **KERNEL** → **SYSTEMD** → **SERVICES** → **APPLICATIONS**

- **HARDWARE**: Physical or virtual server
- **KERNEL**: Linux kernel (manages hardware)
- **SYSTEMD**: Initializes and manages services
- **SERVICES**: Web, database, container, etc.
- **APPLICATIONS**: Your actual workloads

8 JUNIOR ENGINEER TIP

Learn the system, not just commands. Understand how RHEL works, how the pieces fit together, and why things are configured a certain way. This makes you a better engineer and helps you troubleshoot effectively.

REAL SYSTEMS. REAL SKILLS. A STRONGER YOU.

YOUR FIRST RHEL SERVER
LOGIN AND SYSTEM DISCOVERY

Red Hat

Get access. Explore the system. Know what you are working with.

1 CONSOLE vs SSH ACCESS

CONSOLE ACCESS

- Direct access to the server using a keyboard and monitor.
- Useful for initial setup, troubleshooting and recovery.
- Common in physical servers and many cloud providers (e.g. EC2 console).

SSH ACCESS

- Secure remote access over the network.
- Most common method for administering RHEL servers.
- Uses the ssh command.
- More secure than unprotected console access.

Always use SSH key authentication when possible.
Avoid using passwords for regular access.

2 THE RHEL SHELL PROMPT EXPLAINED

```
[user@rhel-server ~]$
```

Username	Hostname	Current Directory	Prompt
Current user (e.g. ec2-user, admin, root)	Name of the server	~ means your home directory	$ normal user # root user (devices/root has full privileges)

The prompt tells you who you are, where you are, and what level of access you have.

3 ESSENTIAL COMMANDS FOR SYSTEM DISCOVERY

Command	What It Does	Example Output (shortened)
whoami	Shows the current user.	ec2-user
hostnamectl	Shows system hostname and OS information.	Static hostname: rhel-server Operating System: Red Hat Enterprise Linux 9.3
cat /etc/redhat-release	Shows the RHEL version.	Red Hat Enterprise Linux release 9.3 (Plow)
uname -r	Shows the kernel version.	5.14.0-362.el9.x86_64
uptime	Shows how long the server has been running.	10:24:03 up 2 days, 3:12, 1 user, load average: 0.05
date	Shows the current date and time.	Tue Sep 2 10:24:03 UTC 2025
timedatectl	Shows date, time and timezone settings.	Time zone: UTC (UTC, +0000)
ip addr	Shows network interfaces and IP addresses.	2: ens5: <BROADCAST,UP> inet 10.0.1.23/24 ...
lsblk	Shows block devices (disks and partitions).	NAME SIZE TYPE MOUNTPOINT sda 50G disk ├─sda1 1G part /boot └─sda2 49G part /

4 BUILDING A QUICK SERVER INVENTORY

- Hostname and operating system
- Kernel version
- Uptime
- Current user and sudo access
- IP address and network interfaces
- Disk and partition information
- CPU and memory (optional: use lscpu, free -h)
- Installed packages (optional: use rpm -qa)

This information helps you understand the server, plan changes, and troubleshoot problems.

5 PRACTICE: IDENTIFY AN UNFAMILIAR RHEL HOST

1. SSH into a RHEL server (or use a lab/VM).
2. Run the commands from this page.
3. Write down the key information (hostname, OS version, IP, disks, etc.).
4. Can you determine how long it has been running and what storage is available?
5. Try this on a different RHEL server and compare the results.

6 JUNIOR ENGINEER TIP

Learn the system, not just commands.
Understanding what the information means will make you a better Linux administrator and help you make the right decisions in production.

"A good engineer knows the system, not just the syntax."

PRACTICE TODAY. CONFIDENCE TOMORROW.

LEARN
PRACTICE
BUILD
GROW

NAVIGATING THE RHEL FILESYSTEM

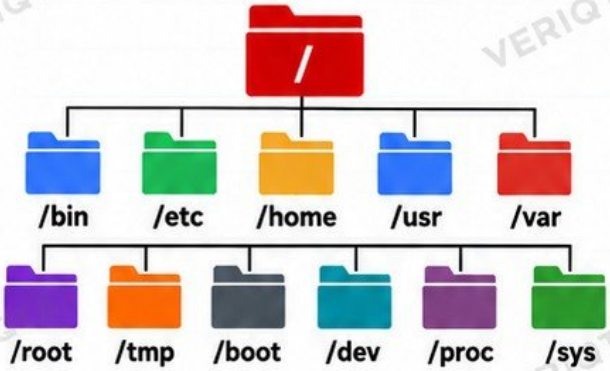
UNDERSTAND THE LINUX DIRECTORY STRUCTURE

Red Hat

Know where things live. Work faster. Troubleshoot better.

1 FILESYSTEM HIERARCHY

RHEL uses the standard Linux filesystem hierarchy. Everything starts from the root directory (/).



The filesystem is like a tree. The root (/) is the top, and everything else is a subdirectory.

2 IMPORTANT DIRECTORIES IN RHEL

Directory	Purpose	Example Contents
/	Root of the filesystem	Everything starts here
/etc	System configuration files	Network, users, services, app configs
/var	Variable data (logs, databases, etc.)	/var/log, /var/lib
/home	User home directories	/home/username
/root	Home directory for root user	Admin files and settings
/usr	User applications and read-only data	/usr/bin, /usr/sbin, /usr/lib
/tmp	Temporary files	Temporary data (usually cleared on reboot)
/boot	Boot loader files and kernel images	vmlinuz, initramfs
/dev	Device files	Disks, terminals, hardware devices
/proc	Virtual filesystem (kernel info)	Process and system information
/sys	Virtual filesystem (device and kernel info)	Hardware and kernel attributes

3 ABSOLUTE vs RELATIVE PATHS

ABSOLUTE PATH

- Starts from the root (/)
- Specifies the full path
- Always points to the same location

Example:
`/etc/fstab`

RELATIVE PATH

- Relative to your current directory
- Shorter and easier to type
- Depends on where you are in the filesystem

Example:
```
../logs
./script.sh
```

4 BASIC COMMANDS FOR NAVIGATION

Command	What It Does	Example
pwd	Shows your current directory	/home/user
ls	Lists files and directories	ls -l
cd	Change directory	cd /etc
cd ..	Go up one directory	cd ..
cd ~	Go to your home directory	cd ~
cd -	Go to previous directory	cd -

5 WHY KNOWING WHERE FILES BELONG MATTERS

- Helps you find and edit configuration files quickly
- Prevents accidental changes to critical system files
- Makes troubleshooting easier
- Helps you follow best practices and Linux standards
- Important for automation (scripts, Ansible, CI/CD)
- Essential in production environments
- Builds confidence as a Linux administrator

The right file in the right place keeps systems stable and makes you a better engineer.

6 JUNIOR ENGINEER TIP

Learn the system, not just systmands.

Understanding the filesystem and where files belong helps you make the right decisions, avoid mistakes, and troubleshoot faster in real production environments.

A STRONG FOUNDATION LEADS TO A STRONGER CAREER.

LEARN
PRACTICE
BUILD
GROW

FILES, DIRECTORIES, LINKS, AND DATA MANAGEMENT

Manage files. Organize data. Work smarter.

Red Hat

Learn Practice Operate Grow

1 ESSENTIAL FILE AND DIRECTORY COMMANDS

Command	What It Does	Example
touch	Create an empty file or update file timestamp	touch file.txt
mkdir	Create a new directory	mkdir mydir
cp	Copy files or directories	cp file.txt /backup/
mv	Move or rename files or directories	mv file.txt /data/ # or rename mv old.txt new.txt
rm	Remove files	rm file.txt
rmdir	Remove empty directories	rmdir mydir

2 LINKS IN LINUX

Command	What It Does	Example
ln	Create links (hard or symbolic)	# Hard link ln file.txt file_hard # Symbolic link ln -s file.txt file_link

HARD LINKS

- Point to the same inode
- Another name for the same file
- Both links share the same data
- If one is deleted, the other still works

Same inode (same data)

SYMBOLIC LINKS (SOFT LINKS)

- Point to the original file path
- Like a shortcut
- Can point to files or directories
- If the original file is deleted, link becomes broken

Points to original file

3 SEARCHING AND FILE INFORMATION

Command	What It Does	Example
find	Search for files and directories	find /var -name "*.log"
locate	Quickly find files using index database	locate sshd_config
file	Show file type	file /etc/passwd
stat	Show detailed file information	stat file.txt

Use locate for speed, but remember to update the database:
sudo updatedb

4 WILDCARDS (PATTERNS)

Pattern	What It Matches	Example
*	Any number of characters	ls *.txt
?	One character	ls file?.txt
[abc]	Any one of the specified characters	ls file[123].txt
[a-z]	Any character in the range	ls file[a-z].txt
{ }	Multiple patterns	ls {*.txt,*.log}

Wildcards help you work faster, especially when dealing with many files.

5 SAFE DELETION PRACTICES

- Always double-check the file or directory path
- Use ls to verify before deleting
- Use rm -i for interactive confirmation
- Be careful with wildcards (e.g. rm *.log)
- Avoid running deletion commands as root
- Consider moving files to a backup location instead of deleting
- Use find with -delete carefully

6 DANGEROUS COMMAND WARNING

BE VERY CAREFUL WITH RECURSIVE DELETION!

This command will permanently delete a directory and everything inside it, without asking:

```
rm -rf /some/directory
```

- There is no undo.
- Files are not moved to a recycle bin.
- A small mistake can delete critical system files.
- Always verify the path before running this command.
- Never run rm -rf / as it can destroy your entire system.

RESPECT THE SYSTEM. SMALL COMMANDS CAN HAVE BIG IMPACT.

READING, EDITING, SEARCHING, AND PROCESSING TEXT

Work with files. Find information. Solve problems.

Red Hat

Learn Practice Operate Grow

1 ESSENTIAL TEXT COMMANDS

Command	What It Does	Example
cat	Display file content	cat file.txt
less	View file content page by page	less /var/log/messages
head	Show first lines	head -n 10 file.txt
tail	Show last lines	tail -n 10 file.txt
grep	Search for text (pattern)	grep error app.log
sort	Sort lines	sort names.txt
uniq	Show unique lines	uniq names.txt
wc	Count lines, words, characters	wc -l file.txt
cut	Extract columns	cut -d':' -f1 /etc/passwd
tee	Write output to file and screen	ls -l | tee output.txt

2 PIPES AND INPUT/OUTPUT REDIRECTION

PIPES (|)

Send the output of one command to another.

Example:
cat app.log | grep error | tail -n 5
Show last 5 error lines from app.log

OUTPUT REDIRECTION (>)

Save output to a file (overwrite).

Example: ls -l > files.txt

APPEND REDIRECTION (>>)

Add output to a file (without overwriting).

Example: echo "New entry" >> notes.txt

INPUT REDIRECTION (<)

Use a file as input for a command.

Example: sort < names.txt

3 EDITING CONFIGURATION FILES

Common editors in RHEL:

- vi / vim (default and common)
- nano (beginner-friendly)

```
# Open a file with vi
vi /etc/ssh/sshd_config

# Open a file with nano
nano /etc/ssh/sshd_config
```

TIPS:

- Always back up files before editing.
- Understand the file format.
- Make small, focused changes.
- Test the service after editing (e.g. systemctl restart sshd).
- Keep notes of what you change.

4 SEARCHING LOGS AND CONFIGURATION

Find a word in a file:

```
grep "keyword" file.log
```

Search recursively in a directory:

```
grep -r "error" /var/log
```

Show line numbers:

```
grep -n "error" app.log
```

Ignore case:

```
grep -i "error" app.log
```

COMMON LOG LOCATIONS:

- /var/log/messages
- /var/log/secure
- /var/log/cron
- /var/log/httpd/
- /var/log/nginx/
- /var/log/maillog
- /var/log/dnf.log

Use grep to quickly find important information in large log files.

5 REAL-WORLD TROUBLESHOOTING PIPELINE

1. **IDENTIFY THE ISSUE** — What are you looking for?
2. **FIND INFORMATION** — Use cat, less, grep, tail, etc.
3. **FILTER AND PROCESS** — Use pipes, sort, uniq, cut, etc.
4. **EDIT IF NEEDED** — Modify config files safely.
5. **VERIFY THE RESULT** — Check logs, restart services, test again.

6 JUNIOR ENGINEER TIP

- Get comfortable reading logs.
- Use grep and less every day.
- Learn to combine commands with pipes.
- Always back up before editing configuration files.
- Practice on real scenarios.
- Text processing skills will make you faster and more effective as a Linux engineer.

FIND. UNDERSTAND. FIX. THAT'S LINUX.

Real Engineers Build Real Skills

USERS, GROUPS, sudo, AND ADMINISTRATIVE ACCESS

Control access. Protect your systems. Work like a professional.

Red Hat

Learn
Practice
Operate
Grow

1 LINUX USERS AND UIDs

- A user is an account that can log in and run commands.
- Each user has a unique User ID (UID).
- UID 0 is the root user (full administrative access).
- Regular users usually start from UID 1000 and above.
- Users have a home directory (e.g. /home/username).

Check a user's details: `id username`

2 GROUPS AND GIDs

- Groups allow you to manage permissions for multiple users.
- Each group has a unique Group ID (GID).
- A user can belong to multiple groups.
- Common groups: root, wheel, users, docker, etc.

Check groups for a user: `groups username`

3 IMPORTANT FILES

File	Purpose
/etc/passwd	Stores user account information
/etc/shadow	Stores encrypted passwords
/etc/group	Stores group information

These files are critical system files. Do not edit them directly. Use the appropriate commands.

4 USER AND GROUP MANAGEMENT COMMANDS

Command	What It Does	Example
useradd	Create a new user	useradd john
usermod	Modify an existing user	usermod -aG wheel john
userdel	Delete a user	userdel john
groupadd	Create a new group	groupadd devops
passwd	Set or change a user's password	passwd john
id	Show user ID and group IDs	id john

5 su AND sudo

Command	What It Does	Example
su	Switch to another user (usually root)	su -
sudo	Run a command as another user (default: root)	sudo systemctl restart httpd

sudo is preferred because it provides accountability and logging. It allows you to perform administrative tasks without sharing the root password.

6 /etc/sudoers AND visudo

- /etc/sudoers controls who can run commands with sudo.
- Always use visudo to edit this file (prevents syntax errors).
- You can allow specific users or groups to run specific commands.

```
# Edit sudoers file safely
sudo visudo

# Example entry (allow user in wheel group)
%wheel  ALL=(ALL)  ALL
```

Never edit /etc/sudoers directly with a regular text editor. Always use visudo.

7 LEAST PRIVILEGE

- Give users only the access they need.
- Use groups to manage permissions.
- Use sudo for administrative tasks.
- Avoid giving full root access to regular users.
- Review access regularly.
- Follow the principle of least privilege.

"Least privilege reduces the risk of accidental changes and security breaches.

8 WHY SHARING ROOT CREDENTIALS IS DANGEROUS

- No accountability (you don't know who did what).
- Higher risk of accidental or malicious changes.
- No audit trail for individual actions.
- Increases the chance of system compromise.
- Violates security best practices and compliance requirements.

Do not share the root password. Use sudo with individual user accounts instead.

9 JUNIOR ENGINEER TIP

- Use your own user account.
- Use sudo for admin tasks.
- Understand users, groups and permissions.
- Always think about security and accountability.
- Good access control is a sign of a professional engineer.

SECURE ACCESS TODAY. A SAFER TOMORROW.

LINUX PERMISSIONS, OWNERSHIP, AND SPECIAL PERMISSIONS

Control access. Secure your system. Prevent problems.

Red Hat
Learn
Practice
Operate
Grow

1 PERMISSION BASICS

Every file and directory has three types of permissions:

Read (r)	Write (w)	Execute (x)
View file content (4)	Modify file content (2)	Run file (as a program or script) (1)

Permissions apply to three classes:

- User (owner)
- Group
- Others (everyone else)

2 VIEWING PERMISSIONS

Use `ls -l` to view file permissions.

```
[user@rhel ~]$ ls -l file.txt
-rw-r--r-- 1 user devops 1024
Sep 10 12:00 file.txt
```

`-rw-r--r--`

- User (rw-)
- Group (r--)
- Others (r--)

The first character (-) indicates a regular file. For directories, it will be d.

3 CHANGING PERMISSIONS (chmod)

Numeric mode:

```
[user@rhel ~]$ chmod 755 script.sh
```

7 = rwx | 5 = r-x | 5 = r-x

Symbolic mode:

```
[user@rhel ~]$ chmod u+x script.sh
```

u = user | g = group | o = others
+ = add | - = remove | = = set exactly

4 SYMBOLIC vs NUMERIC PERMISSIONS

Symbolic (easier to read)

```
[user@rhel ~]$ chmod u+r file.txt
```

- u = user
- g = group
- o = others
- a = all (u,g,o)
- + = add permission
- - = remove permission
- = = set permission

Numeric (faster to use)

```
[user@rhel ~]$ chmod 644 file.txt
```

- 4 = read (r)
- 2 = write (w)
- 1 = execute (x)

Common examples:

- 644 = rw-r--r--
- 755 = rwxr-xr-x
- 600 = rw-------

5 OWNERSHIP (chown and chgrp)

Change file owner:

```
[user@rhel ~]$ chown user file.txt
```

Change group:

```
[user@rhel ~]$ chgrp devops file.txt
```

Change both owner and group:

```
[user@rhel ~]$ chown user:devops file.txt
```

Only the owner or root can change ownership.

6 UMASK

Umask sets default permissions for new files and directories.

```
[user@rhel ~]$ umask
0022
```

Common values:

- 022 → files: 644, dirs: 755
- 077 → files: 600, dirs: 700

Lower umask = more permissive
Higher umask = more restrictive

7 SPECIAL PERMISSIONS

Permission	Symbol	Numeric	What It Does	Example
SUID (Set User ID)	s	4	Run as file owner (even if executed by another user)	chmod u+s /usr/bin/app
SGID (Set Group ID)	s	2	Run as file group (or new files inherit the group)	chmod g+s /shared
Sticky Bit	t	1	Prevent non-owners from deleting files in a directory (common on /tmp)	chmod +t /tmp

8 PERMISSION DENIED? TROUBLESHOOTING STEPS

1. Check file permissions (ls -l).
2. Check file ownership (ls -l).
3. Verify your user and groups (id).
4. Check if the path is correct.
5. Check if the filesystem is mounted read-only.
6. Check SELinux (if enabled).
7. Review application or service requirements.

9 WHY chmod 777 IS WRONG

- Gives everyone full access (read, write, execute).
- Creates serious security risks.
- Can be exploited by malicious users.
- Violates the principle of least privilege.
- Often a sign of poor permission management.
- There is almost always a better, more secure solution.

Use the minimum permissions required to do the job.

10 JUNIOR ENGINEER TIP

- Understand what each permission means.
- Use least privilege.
- Review permissions regularly.
- Don't use chmod 777 to "make it work".
- Learn to read and interpret ls -l output.

SECURE SYSTEMS BUILD BETTER ENGINEERS

"Permissions are not just about access. They are about responsibility."

Real Engineers Build Real Skills

VERIQTA

SOFTWARE MANAGEMENT WITH RPM AND DNF

Red Hat

Install. Update. Manage. Keep Your RHEL System Secure.

1 RPM PACKAGES

- RPM (Red Hat Package Manager) is the package format used by RHEL.
- An RPM package contains software, metadata, dependencies and installation instructions.
- You can install, query, verify and manage RPM packages using the rpm command or dnf (which uses RPM under the hood).

2 PACKAGE REPOSITORIES

- Repositories are online or local locations that store RPM packages.
- DNF connects to repositories to find, download and install packages.
- RHEL uses Red Hat repositories (requires subscription).
- You can also configure third-party or internal repositories.

3 COMMON RPM COMMANDS

Command	What It Does	Example
rpm -q <package>	Check if a package is installed	rpm -q httpd
rpm -qi <package>	Show detailed package info	rpm -qi httpd
rpm -ql <package>	List files installed by a package	rpm -ql httpd
rpm -qf <file>	Find which package owns a file	rpm -qf /etc/httpd/conf/httpd.conf
rpm -e <package>	Remove a package	rpm -e httpd
rpm -ivh <file.rpm>	Install a local RPM file	rpm -ivh package.rpm

4 COMMON DNF COMMANDS

Command	What It Does	Example
dnf search <keyword>	Search for packages	dnf search nginx
dnf info <package>	Show package details	dnf info nginx
dnf install <package>	Install a package	dnf install nginx
dnf remove <package>	Remove a package	dnf remove nginx
dnf update	Update all packages	dnf update
dnf history	Show transaction history	dnf history
dnf history undo <ID>	Undo a previous transaction	dnf history undo 123

5 REPOSITORY CONFIGURATION

- Repository configuration files are stored in /etc/yum.repos.d/
- Each .repo file defines a repository name, base URL, and settings.
- You can enable or disable repositories easily with dnf config-manager.
- Example file: /etc/yum.repos.d/rhel.repo

Example:

```
[rhel-baseos]
name=RHEL 9 BaseOS
baseurl=https://cdn.redhat.com/...
enabled=1
gpgcheck=1
gpgkey=https://.../RPM-GPG-KEY-redhat-release
```

6 GPG VERIFICATION

- GPG (GNU Privacy Guard) verifies that packages are really from Red Hat and have not been tampered with.
- Repositories use a GPG key to sign packages.
- RHEL automatically verifies packages if gpgcheck=1 is enabled.
- You can manually import a key if needed.

Example (import Red Hat GPG key):

```
sudo rpm --import \
 https://www.redhat.com/
 security/data//
 RPM-GPG-KEY-redhat-release
```

7 DEPENDENCIES

- Packages often depend on other packages (libraries, utilities, etc.).
- DNF automatically resolves and installs dependencies.
- You can view dependencies using dnf info or dnf repoquery.
- If a dependency is missing, DNF will show an error.

Example (view dependencies):

```
dnf info nginx
dnf repoquery --requires nginx
```

8 PACKAGE TROUBLESHOOTING

- Package not found? Check your repositories are enabled.
- Dependency errors? Use dnf provides or dnf repoquery.
- Conflicting packages? Use dnf swap or remove the conflict.
- Corrupted package? Reinstall it.
- Check logs in /var/log/dnf.log for more details.

9 PRODUCTION PATCHING AWARENESS

- Regularly update your RHEL systems to get security fixes and bug fixes.
- Use dnf update or targeted updates (dnf update <package>).
- Test updates in a non-production environment first.
- Schedule maintenance windows.
- Review release notes for major updates.
- Keep your system subscribed to Red Hat for security advisories.

JUNIOR ENGINEER TIP

Understand what you install, where it comes from, and why. Package management is not just about commands, it is about keeping your systems secure, stable and reliable in production.

PATCH TODAY A MORE STABLE TOMORROW

VERIQTA

PAGE 9

PROCESSES, JOBS, AND RESOURCE INVESTIGATION

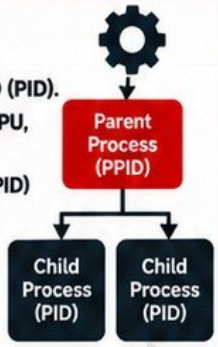
Red Hat

Learn Practice Operate Grow

Understand what is running. Find problems. Keep your systems healthy.

1 WHAT IS A PROCESS?

- A process is a running instance of a program.
- Each process has a unique Process ID (PID).
- Processes use system resources like CPU, memory, and I/O.
- Every process has a parent process (PPID) that started it.
- When a process ends, it is terminated and resources are released.



2 PID AND PPID

- **PID:** Unique number for a process.
- **PPID:** Process ID of the parent process.
- Use these to understand process relationships and troubleshoot issues.

PID	PPID	COMMAND
1	0	systemd
1234	1	sshd
5678	1234	bash
9101	5678	top

3 FOREGROUND vs BACKGROUND JOBS

- **Foreground:** Runs in the current terminal. You see the output.
- **Background:** Runs without blocking your terminal.
- You can move jobs between foreground and background.

```
[user@rhel ~]$ long-task.sh
^Z
[1]+  Stopped     long-task.sh
[user@rhel ~]$ bg %1
[1]+  long-task.sh &
```

4 COMMON PROCESS COMMANDS

Command	What It Does	Example
ps	Show running processes	ps aux
top	Interactive process viewer	top
pgrep	Find process by name	pgrep nginx
jobs	List background jobs	jobs
bg	Resume a job in background	bg %1
fg	Bring a job to foreground	fg %1
kill	Send signal to a process	kill 1234
kill -9	Forcefully kill a process (SIGKILL)	kill -9 1234

5 SIGNALS

- Signals are messages sent to processes.
- Common signals:

Signal	Number	What It Does
SIGTERM	15	Graceful stop (default)
SIGKILL	9	Forceful stop (cannot be ignored)
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl + C)
SIGSTOP	19	Pause process
SIGCONT	18	Continue process

6 CPU AND MEMORY INVESTIGATION

- Use top or htop to view real-time resource usage.
- Check CPU and memory usage with ps, top, or from /proc.
- Identify processes consuming high resources.
- Investigate why (application issue, memory leak, runaway process).

PID	USER	%CPU	%MEM	COMMAND
1892	root	25.3	5.1	java
3241	nginx	12.1	2.3	nginx
5678	user	5.6	1.1	python

7 ZOMBIE PROCESSES

- A zombie process has finished execution but still has an entry in the process table.
- State shows as Z in ps output.
- Usually caused by a parent process not collecting the exit status.
- Check with: `ps aux | grep Z`
- Fix by restarting the parent process or the service.

```
[user@rhel ~]$ ps aux | grep Z
Z          [defunct]
```

8 HIGH-RESOURCE PROCESS TROUBLESHOOTING

1. Identify the process (top, ps, pgrep).
2. Check what it is doing.
3. Review application logs.
4. Look for memory leaks or high CPU loops.
5. Check disk and network activity.
6. Decide: restart, reconfigure, or fix the application.
7. Monitor after making changes.

9 WHY KILL -9 SHOULD NOT BE YOUR FIRST RESPONSE

- kill -9 (SIGKILL) immediately terminates the process.
- It does not allow the process to clean up resources.
- Can cause data corruption, partial writes, or application issues.
- Always try a graceful stop first (kill <PID> or SIGTERM).
- Use kill -9 only when the process ignores normal termination.

10 JUNIOR ENGINEER TIP

Understand what is running and why.
Processes are not just numbers – they are real applications using real resources.
Investigate before you kill. A good engineer finds the root cause, not just a quick fix.

OBSERVE. INVESTIGATE. TROUBLESHOOT. IMPROVE.

systemd AND PRODUCTION SERVICE MANAGEMENT

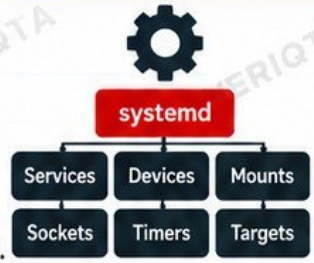
Red Hat

Learn Practice Operate Grow

Manage Services. Keep Systems Running. Build Reliable Infrastructure.

1 WHAT systemd MANAGES

- systemd is the init system and service manager for modern Linux distributions like RHEL.
- It manages the boot process, services, sockets, devices, mounts and more.
- It starts and keeps services running, handles dependencies and restarts failed services.
- It replaces older init systems (e.g. SysVinit).



2 UNITS IN systemd

- Everything in systemd is a unit.
- Common unit types:

Service unit	(.service)	Manages a service
Socket unit	(.socket)	Manages a socket
Device unit	(.device)	Manages hardware devices
Mount unit	(.mount)	Manages mount points
Timer unit	(.timer)	Runs scheduled tasks
Target unit	(.target)	Groups multiple units

3 SERVICE UNITS

- Service units (.service) define how a service starts, stops and runs.
- Unit files are usually located in:

`/usr/lib/systemd/system/` (package provided)

`/etc/systemd/system/` (custom configurations)

- You can create your own service files if needed.

Example service file (nginx.service):

```
[Unit]
Description=NGINX Web Server
After=network.target

[Service]
Type=forking
ExecStart=/usr/sbin/nginx
ExecReload=/bin/kill -s HUP $MAINPID
ExecStop=/bin/kill -s QUIT $MAINPID
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

4 COMMON systemctl COMMANDS

Command	What It Does
systemctl status <service>	Show service status
systemctl start <service>	Start a service
systemctl stop <service>	Stop a service
systemctl restart <service>	Restart a service
systemctl reload <service>	Reload a service (configuration)
systemctl enable <service>	Enable service at boot
systemctl disable <service>	Disable service at boot
systemctl is-active <service>	Check if service is running
systemctl is-enabled <service>	Check if service is enabled at boot

Example:

```
[user@rhel ~]$ sudo systemctl status nginx
[user@rhel ~]$ sudo systenctl start nginx
juser@rhel ~]$ sudo systenctl enable nginx
```

5 UNIT DEPENDENCIES

- Units can depend on other units.
- Common dependencies:
  - After= Start after a unit
  - Requires= Requires another unit
  - Wants= Suggests another unit
  - Before= Start before a unit

Example (service with dependency):

```
[Unit]
Description=My App
After=network.target
Requires=network.target
```

6 FAILED SERVICES

- A service can fail to start due to:
  - Incorrect configuration
  - Missing dependencies
  - Permission issues
  - Resource problems
  - Application errors
- Check the status and logs:

```
systemctl status <service>
journalctl -u <service> -xe
```

7 SERVICE STARTUP TROUBLESHOOTING

1. Check service status: systemctl status <service>
2. Read logs: journalctl -u <service> -xe
3. Verify configuration files (e.g. /etc/<app>/<app>.conf)
4. Check dependencies: systemctl list-dependencies <service>
5. Confirm required packages are installed
6. Check permissions (files, directories, ports)
7. Look for resource issues (CPU, memory, disk space)
8. Fix the issue and restart the service

8 PRACTICAL EXAMPLE: NGINX SERVICE

```
[user@rhel ~]$ sudo systemctl status nginx
[user@rhel ~]$ sudo systemctl start nginx
[user@rhel ~]$ sudo systemctl enable nginx
[user@rhel ~]$ sudo systemctl is-active nginx
active
[user@rhel ~]$ sudo systemctl is-enabled nginx
enabled
```

Service is running and enabled at boot.

9 JUNIOR ENGINEER TIP

- Always check the service status and logs first.
- Understand the unit dependencies.
- Learn how your service is configured.
- Make small changes, test and verify.
- Good service management keeps production stable.

STABLE SERVICES. RELIABLE SYSTEMS. BETTER ENGINEERS.

LOGS, JOURNALD, AND FINDING WHAT FAILED

Red Hat

Logs tell the story. Find the problem. Fix it faster.

Learn Practice Operate Grow

1 WHY LOGS MATTER

- Logs record what happens on your system.
- They help you understand normal behaviour and find the cause of failures.
- Without logs, troubleshooting is just guessing.
- In production, logs save time, reduce downtime and provide evidence.

Good engineers read logs.
Great engineers understand the story behind the logs.

2 SYSTEMD JOURNAL

- systemd stores logs in a binary log system called journald.
- journald collects logs from the kernel, system services and applications.
- It replaces many traditional log files and allows powerful filtering.
- Logs are stored in /var/log/journal/ (persistent) or in memory (depending on configuration).

3 JOURNALCTL BASICS

Command	What It Does
journalctl	View all logs
journalctl -u <service>	View logs for a specific service
journalctl -b	View logs for current boot
journalctl -k	View kernel logs
journalctl -f	Follow logs in real time
journalctl --since "1 hour ago"	Show logs from a specific time
journalctl --until "2025-08-01 10:00"	Show logs until a specific time

4 COMMON JOURNALCTL COMMANDS

Command	Example	Description
journalctl	journalctl	Show all logs
journalctl -u	journalctl -u nginx	Logs for a service
journalctl -b	journalctl -b	Logs since last boot
journalctl -f	journalctl -f	Follow logs live
journalctl --since	journalctl --since "2 hours ago"	Logs from a time
journalctl --until	journalctl --until "2025-08-01 12:00"	Logs until a time
journalctl -p	journalctl -p err	Filter by priority (err, warning, info, etc.)
journalctl --no-pager	journalctl --no-pager -u sshd	Output without pager

5 /VAR/LOG AND TRADITIONAL LOG FILES

- Many applications still write logs to files in /var/log.
- Common log files:

/var/log/messages	(general system logs)
/var/log/secure	(authentication logs)
/var/log/mailog	(mail server logs)
/var/log/cron	(cron jobs)
/var/log/httpd/	(Apache logs)
/var/log/nginx/	(Nginx logs)
/var/log/yum.log	(package management)

- Use less, tail or grep to read log files.

6 LOG ROTATION

- Log files can grow very large.
- logrotate automatically rotates, compresses and removes old log files.
- Configuration files are in /etc/logrotate.conf and /etc/logrotate.d/.
- This prevents disks from filling up and keeps systems stable.

Always check log rotation for custom applications in production.

7 FOLLOWING LIVE LOGS

- Use journalctl -f to follow logs in real time.
- Useful when troubleshooting a service during startup or while reproducing an issue.
- You can also use tail -f for log files.

```
[user@rhel ~]$ journalctl -u nginx -f
Aug 01 10:24:13 rhel nginx[1234]: Starting nginx...
Aug 01 10:24:13 rhel nginx[1234]: nginx started successfully.
Aug 01 10:25:01 rhel nginx[1234]: New connection from 192.168.1.10
```

8 CONNECTING SERVICE STATUS TO LOGS

- When a service fails, always check its logs.
- Use systemctl status <service> first.
- Then use journalctl -u <service> to see the detailed logs.
- This shows the exact error and why the service failed.

```
[user@rhel ~]$ systemctl status nginx
[user@rhel ~]$ journalctl -u nginx --no-pager
```

9 TROUBLESHOOTING MODEL

- Symptom → What is the issue?
- Status → Check service/system status
- Logs → Read relevant logs
- Evidence → Find the exact error or clue
- Cause → Identify the root cause
- Fix → Apply the correct fix
- Verify → Confirm it is working

10 JUNIOR ENGINEER TIP

- Always check the logs, even if the service looks like it is running.
- Logs tell you what really happened.
- Get comfortable using **journalctl** and reading log files.
- In production, the ability to quickly find and understand logs makes you a valuable engineer.

GOOD LOGS.
FASTER SOLUTIONS.
STRONGER ENGINEERS.

LEARN PRACTICE OPERATE GROW

RHEL NETWORKING FOR SYSTEM ADMINISTRATORS

Understand. Configure. Troubleshoot. Keep Systems Connected.

Red Hat

Learn Practice Operate Grow

1 NETWORKING BASICS

- **Interfaces** – Network cards that connect your server to a network.
- **IP addresses** – Unique address for a device.
- **Subnets** – Divide networks into smaller ranges.
- **Default gateway** – Router that sends traffic to other networks.
- **DNS** – Converts domain names to IP addresses.
- **Routes** – Tell the server where to send traffic.
- **Ports** – Communication endpoints for services (e.g. 22 for SSH, 80 for HTTP).

2 VIEW NETWORK INTERFACES

```
[user@rhel ~]$ ip addr
1: lo: <LOOPBACK,UP> mtu 65536 ...
   inet 127.0.0.1/8 scope host lo
2: ens160: <BROADCAST,MULTICAST,UP> ...
   inet 192.168.1.10/24 brd 192.168.1.255
   valid_lft forever preferred_lft forever
```

Shows all network interfaces and their IP addresses.

3 USEFUL IP COMMANDS

Command	What It Does
ip addr	Show IP addresses on interfaces
ip link	Show interface status (UP/DOWN)
ip route	Show/modify routing table
ip route add <route>	Add a static route
ip route del <route>	Delete a static route

4 CHECK CONNECTIVITY

Command	What It Does	Example
ping	Test network connectivity	ping 8.8.8.8
curl	Test HTTP/HTTPS connectivity	curl -I https://veriqta.com
dig	Query DNS records	dig veriqta.com
ss	Show listening ports and connections	ss -tulnp

5 DNS AND NAME RESOLUTION

```
[user@rhel ~]$ dig veriqta.com
;; ANSWER SECTION:
veriqta.com.    300  IN      A
104.21.42.10
veriqta.com.    300  IN      A
172.67.198.12
```

DNS converts domain names to IP addresses.

6 NETWORKMANAGER (nmcli)

Command	What It Does
nmcli device status	Show device status
nmcli connection show	List connections
nmcli connection up <name>	Activate a connection
nmcli connection down <name>	Deactivate a connection
nmcli device show	Show device details

7 PERSISTENT NETWORK CONFIGURATION

- Use NetworkManager (nmcli) for persistent configuration.
- Network settings are stored in /etc/NetworkManager/system-connections/
- You can also configure static IP, gateway and DNS using nmcli:

```
nmcli connection modify ens160 \\
  ipv4.addresses 192.168.1.10/24 \
  ipv4.gateway 192.168.1.1
  ipv4.dns "8.8.8.8 1.1.1.1"
  ipv4.method manual
nmcli connection up ens160
```

8 COMMON PORTS

Port	Service	Description
22	SSH	Secure remote access
80	HTTP	Web traffic (not encrypted)
443	HTTPS	Web traffic (encrypted)
53	DNS	Domain name resolution
25	SMTP	Email sending
3306	MySQL	Database (MySQL)
5432	PostgreSQL	Database (PostgreSQL)
6379	Redis	In-memory database
8080	HTTP (Alt)	Alternative web port

9 CONNECTION ERRORS

CONNECTION REFUSED

The target host is reachable but no service is listening on the port.

- Check if the service is running.
- Example: `curl http://192.168.1.10:8080`

CONNECTION TIMEOUT

The request did not get a response.

- Possible causes: firewall, network issue, wrong IP, or the host is down.
- Example: `ping 10.0.0.1`

DNS FAILURE

- The domain name cannot be resolved to an IP address.
- Check DNS configuration and internet access.
- Example: `dig veriqta.com`

10 TROUBLESHOOTING STEPS

1. Check interface status (ip addr, nmcli device status)
2. Verify IP address, subnet and gateway
3. Test connectivity (ping, curl)
4. Check DNS resolution (dig, nslookup)
5. Check routes (ip route)
6. Verify listening ports (ss -tulnp)
7. Review firewall rules (firewall-cmd --list-all)
8. Check service status if the port is open

11 JUNIOR ENGINEER TIP

- Always start from the network layer when a service is not reachable.
- Use simple tools like ping, curl and dig before checking complex configurations.
- Understand the difference between "refused", "timeout" and "DNS failure".
- Good network troubleshooting skills will save you many hours in production.

A CONNECTED INFRASTRUCTURE KEEPS BUSINESS RUNNING.

FIREWALLD AND HOST NETWORK SECURITY

Red Hat

Secure
Configure
Monitor
Grow

Control Access. Allow What You Need. Keep Your RHEL Systems Secure.

1 WHY HOST FIREWALLS MATTER

- A host firewall controls incoming and outgoing network traffic to your server.
- Helps block unauthorized access.
- Reduces the attack surface.
- Essential even in cloud environments.
- Works alongside network firewalls and security groups.
- Helps you enforce the principle of least privilege.

Do not rely only on cloud security groups. Always secure the host itself.

2 WHAT IS FIREWALLD?

- firewalld is the default firewall management tool in RHEL.
- Dynamically manages firewall rules.
- Uses zones, services and ports.
- Supports runtime and permanent configuration.
- Allows easy integration with system services.
- Replaces the older iptables firewall management.

3 ZONES

- A zone defines the level of trust for network connections.
- Each zone has its own rules.
- Common zones:

Zone	Use Case
public	Default for external networks
internal	Trusted internal networks
dmz	Systems in a demilitarized zone
work	Workplace networks
home	Home networks
trusted	Fully trusted (no restrictions)
drop	Drops all incoming traffic
block	Rejects incoming traffic with an error message

4 SERVICES

- A service is a predefined set of ports and protocols.
- Using services is easier and safer than opening individual ports.
- Example services:

Service	Description
ssh	Secure Shell (port 22)
http	Web server (port 80)
https	Secure web server (port 443)
dns	Domain Name System (port 53)
ftp	File Transfer Protocol (port 21)
smtp	Email (port 25)

5 PORTS

- You can open specific ports instead of using a service.
- Specify port and protocol.
- Example: 8080/tcp

Syntax:

```
--add-port=<port>/<protocol>
```

Example:

```
sudo firewall-cmd \
  --add-port=8080/tcp \
  --permanent
```

6 RUNTIME vs PERMANENT

- Runtime: changes take effect immediately but are lost after a reboot.
- Permanent: changes are saved and persist after a reboot.

Best practice: use --permanent, then reload the firewall.

7 COMMON FIREWALL-CMD COMMANDS

Task	Command
Check firewall status	`sudo firewall-cmd --state`
List active zone	`sudo firewall-cmd --get-active-zones`
List all zones	`sudo firewall-cmd --get-zones`
List services in zone	`sudo firewall-cmd --zone=public --list-services`
List open ports	`sudo firewall-cmd --zone=public --list-ports`
Allow a service	`sudo firewall-cmd --add-service=http --permanent`
Open a port	`sudo firewall-cmd --add-port=8080/tcp --permanent`
Remove a service	`sudo firewall-cmd --remove-service=http --permanent`
Reload configuration	`sudo firewall-cmd --reload`

8 VERIFYING LISTENING SOCKETS

Use ss to check which ports are listening on your server.

```
sudo ss -tuln
sudo ss -tuln | grep :80
```

- `-t` : TCP connections
- `-u` : UDP connections
- `-l` : Listening ports
- `-n` : Show numeric addresses

9 TROUBLESHOOTING BLOCKED APPLICATION TRAFFIC

- Confirm the service is running (systemctl).
- Check if the firewall allows the port/service (firewall-cmd).
- Verify the application is listening (ss).
- Test from another server (curl, nc, telnet).
- Check the correct zone is being used.
- Look at logs (/var/log/messages or journalctl).
- Ensure no other firewall (cloud security group or network ACL) is blocking traffic.

10 PRODUCTION WARNING

- Do not open unnecessary ports or services.
- Only allow what is required for your application and administration.
- Regularly review firewall rules.
- Remove unused services and ports.
- Keep documentation of all firewall changes.
- Follow your organization's security policy.

Unnecessary open ports increase your attack surface and can lead to serious security breaches.

11 JUNIOR ENGINEER TIP

Always understand why you are opening a port or allowing a service. If you are not sure, ask. It is better to be safe than to expose your server.

**SECURE TODAY.
A MORE STABLE TOMORROW.**

STORAGE, PARTITIONS, FILESYSTEMS, AND MOUNTS

Red Hat

Learn Practice Operate Grow

Manage Disk Space. Keep Your RHEL Systems Running.

1 DISKS AND BLOCK DEVICES

- Disks are accessed as block devices (e.g. /dev/sda, /dev/nvme0n1).
- A disk can be divided into partitions, formatted with a filesystem, and mounted at a directory.
- Use lsblk and blkid to view available disks and their details.

2 USEFUL COMMANDS

Command	What It Does
lsblk	List block devices (disks, partitions, mount points)
blkid	Show filesystem type and UUID
df -h	Show mounted filesystems and disk usage
du -sh <dir>	Show directory size
mount	Mount a filesystem
umount	Unmount a filesystem

3 PARTITIONS

- A disk can be divided into partitions (e.g. /dev/sda1, /dev/sda2).
- Each partition can have a filesystem.
- Use tools like fdisk, parted or lsblk to view and manage partitions.
- Choose the right partition scheme (MBR or GPT).

4 FILESYSTEMS

- A filesystem organizes how data is stored on a partition.
- RHEL commonly uses XFS (default) and also supports ext4.
- Choose a filesystem based on your use case and performance needs.

Filesystem	Key Features
XFS	• Default on RHEL • High performance • Good for large files and large volumes • Online resizing
ext4	• Widely used • Stable and reliable • Good for general purpose use

5 MOUNT POINTS

- A mount point is a directory where a filesystem is attached (e.g. /, /data).
- Use mount to attach a filesystem.
- Use umount to detach it.
- Create the mount point directory before mounting.

```
sudo mkdir /data
sudo mount /dev/sdb1 /data
sudo umount /data
```

6 /etc/fstab

- /etc/fstab defines filesystems that should be automatically mounted at boot.
- Use UUIDs instead of device names to avoid issues when device names change.
- Always test your fstab changes before rebooting.

```
# Example /etc/fstab entry
UUID=123e4567-e89b-12d3-a456-426614174000
/data          xfs     defaults    0 2
```

7 UUIDs

- A UUID (Universally Unique Identifier) is a unique ID for a filesystem.
- Use blkid to find the UUID.
- UUIDs are more reliable than device names in /etc/fstab.

```
sudo blkid
# Example output
/dev/sdb1: UUID="123e4567-e89b-12d3-a456-426614174000"
TYPE="xfs"
```

8 PERSISTENT MOUNTING

- Add an entry to /etc/fstab using the UUID.
- Test the configuration:

```
sudo mount -a
```

- If there are no errors, the filesystem will be mounted automatically at boot.
- Verify with:

```
df -h
```

9 DISK-FULL TROUBLESHOOTING

- Use df -h to find which filesystem is full.
- Use du -sh * to find large directories.
- Check for large logs in /var/log.
- Clean up old files, logs or unused packages.
- Consider expanding the filesystem if needed.

```
sudo du -sh /var/* | sort -h
```

10 PREVENTING BOOT PROBLEMS

- Incorrect /etc/fstab entries can prevent the system from booting.
- Use the correct UUID and filesystem type.
- Test with sudo mount -a before rebooting.
- Use the nofail option for non-critical filesystems.
- Ensure critical filesystems (/, /boot) are properly configured.
- Keep access to the server (console, cloud console or rescue mode) in case of boot issues.

11 JUNIOR ENGINEER TIP

- Always understand your storage layout, what is mounted where, and why.
- A well-planned storage configuration keeps your system stable and avoids production outages.

RIGHT STORAGE. HIGHER UPTIME. A MORE RELIABLE YOU.

LEARN PRACTICE OPERATE GROW

VERIQTA

LVM (LOGICAL VOLUME MANAGER)

MANAGING STORAGE WITHOUT STARTING OVER

Flexible Storage. More Control. Built for Production.

Red Hat

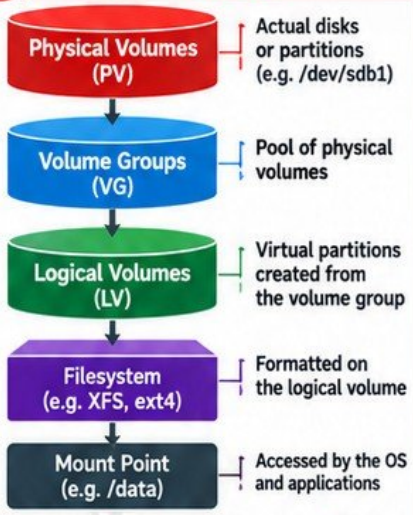
Learn Practice Operate Grow

1 WHY LOGICAL VOLUME MANAGER EXISTS

- LVM provides a flexible way to manage storage on Linux (including RHEL).
- Allows you to resize, extend and manage storage without re-partitioning or reinstalling the OS.
- Separates the physical storage from the logical storage, making it easy to grow and adapt as needs change.
- Widely used in enterprise environments for scalability and high availability.

LVM = More flexibility, less downtime, better storage management.

2 LVM ARCHITECTURE



PV → VG → LV → Filesystem → Mount

3 KEY CONCEPTS

Component	Description
Physical Volume (PV)	A physical disk or partition initialized for LVM (e.g. /dev/sdb1).
Volume Group (VG)	A pool of one or more physical volumes.
Logical Volume (LV)	A virtual partition created from the volume group.
Filesystem	Formatted on the logical volume (e.g. XFS or ext4).
Mount Point	The directory where the filesystem is attached (e.g. /data).

You can add more physical disks to a volume group and extend your logical volumes without starting over.

4 USEFUL LVM COMMANDS

Command	What It Does
pvs	Show physical volumes
vgs	Show volume groups
lvs	Show logical volumes
pvcreate /dev/sdb1	Initialize a disk/partition as a PV
vgcreate vg_data /dev/sdb1	Create a volume group
lvcreate -n lv_data -L 10G vg_data	Create a logical volume
lvextend -L +10G /dev/vg_data/lv_data	Extend a logical volume
resize2fs /dev/vg_data/lv_data	Grow an ext4 filesystem
xfs_growfs /data	Grow an XFS filesystem

5 CREATING LOGICAL VOLUMES (EXAMPLE)

```
# 1. Initialize a disk/partition as a PV
sudo pvcreate /dev/sdb1
# 2. Create a volume group
sudo vgcreate vg_data /dev/sdb1
# 3. Create a logical volume
sudo lvcreate -n lv_data -L 10G vg_data
# 4. Format with XFS (recommended for RHEL)
sudo mkfs.xfs /dev/vg_data/lv_data
# 5. Create mount point and mount
sudo mkdir /data
sudo mount /dev/vg_data/lv_data /data
# 6. Verify
df -h | grep /data
```

6 EXTENDING STORAGE

```
# Extend logical volume by 10GB
sudo lvextend -L +10G
/dev/vg_data/lv_data

# For XFS (grow directly)
sudo xfs_growfs /data

# For ext4 (use resize2fs)
sudo resize2fs
/dev/vg_data/lv_data

# Verify new size
df -h | grep /data
```

7 GROWING FILESYSTEMS

- XFS: Use xfs_growfs (can be grown online).
- ext4: Use resize2fs (can also be grown online).
- Always ensure the logical volume is extended before growing the filesystem.
- Verify the new size with df -h.

XFS is the default filesystem for RHEL and is recommended for most use cases.

8 CAPACITY PLANNING

- Monitor disk usage regularly (df -h, vgs, lvs).
- Plan for future growth.
- Keep enough free space in the volume group.
- Consider using multiple disks in a volume group.
- Set up monitoring and alerts for disk usage.

9 PRODUCTION PRECAUTIONS

- Always back up important data before making storage changes.
- Verify you are extending the correct logical volume.
- Ensure the filesystem type matches the correct grow command.
- Avoid removing physical volumes that are in use.
- Test changes in a non-production environment first.
- Have a rollback plan in case of issues.

10 JUNIOR ENGINEER TIP

Understand the LVM structure and always verify before making changes. LVM gives you flexibility, but mistakes can lead to data loss.

PLAN YOUR STORAGE. KEEP YOUR SYSTEMS RUNNING.

LEARN PRACTICE OPERATE GROW

SELinux WITHOUT DISABLING IT

Stronger Security, Better Control. Production Ready.

Red Hat

Learn Practice Operate Grow

1 WHAT SELINUX PROTECTS

- SELinux (Security-Enhanced Linux) protects your system from unauthorized access and limits the damage if a service is compromised.
- It enforces security policies on processes, files, and network resources.
- Helps prevent misconfigurations and stops many attacks, even if a user has root access.

SELinux adds an extra layer of protection beyond normal Linux permissions.

2 MANDATORY ACCESS CONTROL

- SELinux uses Mandatory Access Control (MAC).
- Unlike traditional Linux permissions (discretionary access control), MAC enforces rules defined by security policy.
- Every process and file has a security label (context).
- Access is allowed or denied based on these labels and policies, not just the user's permissions.

Subject (process) → Policy rules → Object (file/resource)

Even root must follow SELinux policy (when enforcing).

3 SELINUX MODES

Mode	What It Means
Enforcing	Actively enforces policy. Blocks unauthorized actions. This is the recommended production mode.
Permissive	Does not block actions, but logs what would have been denied. Useful for troubleshooting.
Disabled	SELinux is completely turned off. No policy is enforced. Not recommended for production.

Do not disable SELinux in production. Learn to work with it.

4 LABELS AND CONTEXTS

- Every file, directory, process and resource has a security context (label).
- A context includes:

```
user:role:type:level
```

Example:

```
system_u:object_r:httpd_sys_content_t:s0
```

- These labels tell SELinux what the object is and how it can be used.
- Use `ls -Z` to view file contexts.

5 USEFUL COMMANDS

Command	What It Does
`getenforce`	Show current SELinux mode
`sestatus`	Show detailed SELinux status
`ls -Z <file>`	Show security context of a file
`restorecon <file>`	Restore default context
`restorecon -Rv <dir>`	Recursively restore contexts in a directory
`semanage`	Manage SELinux policy (configuration)

6 SELINUX BOOLEANS

- SELinux booleans allow you to enable or disable specific policy features.
- View all booleans:

```
getsebool -a
```

- Check a specific boolean:

```
getsebool httpd_can_network_connect
```

- Enable a boolean (temporary):

```
setsebool httpd_can_network_connect on
```

- Enable a boolean (permanent):

```
setsebool -P httpd_can_network_connect on
```

7 DIAGNOSING SELINUX DENIALS

- When SELinux blocks an action, it logs a denial message (AVC - Access Vector Cache).
- View recent denials:

```
sudo ausearch -m AVC -ts recent
```

- View denials in audit log:

```
sudo less /var/log/audit/audit.log
```

- Use `audit2why` to understand the cause:

```
sudo audit2why -m AVC -ts recent
```

- Use `audit2allow` to generate a custom policy (if needed):

```
sudo audit2allow -m AVC -ts recent
```

8 WHY setenforce 0 IS NOT A FIX

setenforce 0

- `setenforce 0` only sets SELinux to permissive temporarily.
- It does not fix the real problem.
- The issue will still exist when SELinux is enforcing.
- It can hide security issues and leave your system exposed.
- Learn to read the logs, understand the denial, and apply the correct fix.

Real engineers fix the root cause, not just disable security.

9 MENTAL MODEL

Normal permissions + SELinux policy

User/Process (e.g. Apache)
↓
Linux Permissions (read, write, execute)
+
SELinux Policy (labels, rules, contexts)
↓
Access Allowed or Denied

Both normal permissions AND SELinux policy must allow access for an operation to succeed.

Think of SELinux as a security layer that works with, not instead of, Linux permissions.

10 JUNIOR ENGINEER TIP

- Keep SELinux enabled in labs and learn how to work with it.
- Understand labels, read the logs, and apply the correct fix.
- Disabling SELinux is easy, but it's not the engineer's way.

SECURE SYSTEMS. STRONGER CAREERS. VERIQTA.

LEARN PRACTICE OPERATE GROW

SSH ADMINISTRATION AND SECURE REMOTE ACCESS

Red Hat

Learn Practice Operate Grow

Secure Connections. Protect Your Servers. Keep Control.

1 SSH CLIENT AND SERVER

- SSH (Secure Shell) provides encrypted remote access to Linux servers.
- SSH client is used to connect to a remote server.
- SSH server (sshd) runs on the remote server and accepts connections.
- All communication is encrypted to protect your credentials and data.

2 sshd (SSH SERVER)

- sshd is the SSH daemon (service) that listens for incoming connections (usually on port 22).
- Check status:

```
sudo systemctl status sshd
```

- Start/Enable SSH:

```
sudo systemctl start sshd
sudo systemctl enable sshd
```

- Check listening port:

```
sudo ss -tuln | grep :22
```

3 PASSWORD vs KEY AUTHENTICATION

Method	Pros	Cons
Password authentication	Easy to set up	Less secure, can be brute forced
Key authentication	More secure, no password needed	Slightly more setup, need to manage keys

Use key authentication in production. Disable password authentication where possible.

4 PUBLIC/PRIVATE KEY MODEL

Public Key (share)

Private Key (keep secret)

- A key pair consists of a public key and a private key.
- You keep the private key on your local machine.
- You copy the public key to the remote server (in authorized_keys).
- The server verifies your private key using the public key.
- No password is sent over the network.

5 GENERATING SSH KEYS

- Generate a new key pair:

```
ssh-keygen -t ed25519 -C "your_email"
```

- You can also use RSA:

```
ssh-keygen -t rsa -b 4096 -C "your_email"
```

- Follow the prompts (choose a location or use the default ~/.ssh/id_ed25519).
- You can add a passphrase for extra security.

6 CONNECTING, SCP AND SFTP

Task	Command	Example
Connect via SSH	ssh <user>@<host>	ssh ec2-user@192.168.1.10
Copy file to server	scp <file> <user>@<host>:<path>	scp app.tar ec2-user@server:/tmp
Copy file from server	scp <user>@<host>: :<path> <file>	scp ec2-user@server: /var/log/app.log .
File transfer (interactive)	sftp <user>@<host>	sftp ec2-user@server

7 ~/.ssh DIRECTORY

```
~/.ssh/
├── id_ed25519        (private key)
├── id_ed25519.pub    (public key)
├── known_hosts       (known servers)
└── authorized_keys   (on server side)
```

- ~/.ssh stores your SSH keys and configuration files.
- Keep your private key safe and never share it.

8 AUTHORIZED_KEYS (ON SERVER)

- The public key must be added to the user's ~/.ssh/authorized_keys on the server.
- Create the directory (if needed):

```
mkdir -p ~/.ssh
chmod 700 ~/.ssh
```

- Add your public key:

```
cat ~/.ssh/id_ed25519.pub >> ~/.ssh/authorized_keys
```

- Set correct permissions:

```
chmod 600 ~/.ssh/authorized_keys
```

- You can add multiple public keys (for multiple machines or users).

9 PRIVATE-KEY PERMISSIONS

- Your private key file must have strict permissions.
- Recommended:

```
chmod 600 ~/.ssh/id_ed25519
```

- If permissions are too open, SSH will refuse to use the key.
- Keep your private key secure. Do not share it or commit it to Git.

10 sshd_config (IMPORTANT SETTINGS)

```
# Example configuration (/etc/ssh/sshd_config)
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AuthorizedKeysFile    .ssh/authorized_keys
AllowUsers devops
MaxAuthTries 3
```

- Use sudo to edit: `sudo vi /etc/ssh/sshd_config`
- After changes, restart SSH: `sudo systemctl restart sshd`

11 ROOT LOGIN CONSIDERATIONS

- Avoid direct root login (PermitRootLogin no).
- Use a regular user with sudo privileges instead.
- Reduces the risk of brute force attacks and accidental damage.
- If root login is required, use key authentication only.
- Monitor and log all administrative access.

12 TROUBLESHOOTING PERMISSION DENIED (PUBLICKEY)

- Check that the correct private key is used.
- Verify the public key is in ~/.ssh/authorized_keys on the server.
- Check file and directory permissions.
- Ensure the SSH service is running.
- Use verbose mode for more details:

```
ssh -v user@server
```

- Check server logs: `sudo journalctl -u sshd`

13 PROTECTING ADMINISTRATIVE ACCESS

- Use key-based authentication.
- Disable password authentication.
- Restrict root login.
- Use strong passphrases.
- Limit access with AllowUsers or IP restrictions.
- Regularly review SSH logs.
- Use tools like fail2ban to block brute force attempts.
- Keep your SSH keys and servers updated.
- Follow the principle of least privilege.

14 JUNIOR ENGINEER TIP

Set up SSH keys, disable password authentication, and secure your SSH configuration early. It will save you time, prevent attacks, and keep your systems safe.

VERIQTA

LEARN PRACTICE OPERATE GROW

SCHEDULING, AUTOMATION, AND ROUTINE ADMINISTRATION

Red Hat

Same Systems Stronger Engineers

Automate Today, More Reliable Systems Tomorrow.

1 WHY ADMINISTRATORS AUTOMATE

- Reduces manual work and human error.
- Saves time and ensures consistency.
- Keeps systems reliable and available.
- Allows you to focus on more important tasks.
- Automation is a core skill for Linux administrators and DevOps engineers.

If you find yourself doing the same task twice, automate it.

2 SHELL SCRIPTS

- Shell scripts help you automate repetitive tasks.
- Use a text editor to create a script.

```
#!/bin/bash
# Example script
echo "Starting system check..."
date
echo "Check complete."
```

- Make it executable:

```
chmod +x script.sh
```

- Run the script:

```
./script.sh
```

3 CRON AND CRONTAB

- cron runs scheduled tasks in the background.
- Use crontab to manage your personal scheduled jobs.

Common crontab commands

Command	What It Does
`crontab -e`	Edit your crontab
`crontab -l`	List your scheduled jobs
`crontab -r`	Remove all jobs

Crontab syntax (5 fields)

Field	Value	Meaning
*	0-59	Minute
*	0-23	Hour
*	1-31	Day of month
*	1-12	Month
*	0-7	Day of week (0 or 7 = Sunday)

Example: Run a script every day at 2 AM

```
0 2 * * * /path/to/script.sh
```

4 SYSTEMD TIMERS

- systemd timers are more powerful and flexible than cron.
- They integrate with the systemd service manager.
- Useful for complex schedules and better logging.

Common systemd timer commands

Command	What It Does
`systemctl list-timers`	List all timers
`systemctl enable <timer>`	Enable a timer
`systemctl start <timer>`	Start a timer
`systemctl status <timer>`	Check timer status

Use systemd timers for better control, monitoring and logging.

5 ENVIRONMENT DIFFERENCES IN SCHEDULED JOBS

- Scheduled jobs (cron or systemd) run in a minimal environment.
- They may not have the same PATH, user variables or shell settings as your interactive session.
- Always use full paths to commands and scripts.

A script that works manually may fail in a scheduled job due to environment differences.

6 REDIRECTING OUTPUT

- Redirect output to a file to keep logs.

```
./script.sh > /var/log/script.log 2>&1

# Append instead of everwrite
./script.sh >> /var/log/script.log 2>&1
```

7 LOGGING AUTOMATION

- Always log the output of automated tasks.
- Use a consistent log location (e.g. /var/log).
- Include timestamps in your logs.

```
echo "$(date): Backup completed" >> /var/log/backup.log
```

8 EXIT CODES

- Every command returns an exit code.
- 0 = success, non-zero = error.
- Use exit codes to detect failures in scripts.

```
if command -v nginx &> /dev/null; then
  echo "Nginx is installed"
else
  echo "Nginx is not installed"
fi
```

9 IDEMPOTENT THINKING

- Automation should be safe to run multiple times without causing problems.
- Example: creating a directory that already exists should not fail.
- Use checks in your scripts.

```
if [ ! -d /backup ]; then
   mkdir -p /backup
fi
```

10 BACKUPS AND MAINTENANCE TASKS

- Common automated tasks:
  - Back up important files
  - Clean up old logs
  - Monitor disk space
  - Update packages
  - Restart services if needed

Automate routine maintenance to keep systems healthy and avoid surprises.

11 AUTOMATION FAILURE TROUBLESHOOTING

- Check logs for error messages.
- Verify permissions and paths.
- Test the script manually.
- Check exit codes.
- Ensure required environment variables are set.
- Use absolute paths.

12 DEVOPS CONNECTION

RHEL → Bash → Ansible → CI/CD

- Start with shell scripts on RHEL.
- Use Ansible for larger automation.
- Integrate with CI/CD pipelines for full automation and infrastructure as code.

13 JUNIOR ENGINEER TIP

- Start small. Automate simple tasks first.
- Always test your automation.
- Good automation makes you a more valuable engineer.

AUTOMATE THE BORING.
BUILD A BETTER TOMORROW.

LEARN PRACTICE OPERATE GROW

VERIQTA

LEARN PRACTICE OPERATE GROW

RHEL PRODUCTION TROUBLESHOOTING AND RECOVERY

Red Hat

Same Systems Stronger Engineers

Find the Cause. Fix the Problem. Keep Production Running.

1 MINDSET: START WITH SYMPTOMS

- Start with symptoms, not assumptions.
- What changed? (recent deployments, configurations, updates, etc.)
- Check logs and gather evidence.
- Avoid making multiple changes at once.
- Follow a structured troubleshooting process.

Be calm, methodical and objective.

2 COMMON PROBLEMS AND SYMPTOMS

Problem	Common Symptoms
Service unavailable	Application or website not responding
High CPU	Slow performance, high load average
High memory	System becomes slow or unresponsive
Disk full	No space left, services failing
Permission denied	Access denied errors
DNS failure	Cannot resolve hostnames
Port unreachable	Connection refused or timeout
Failed service	Service not starting or keeps stopping
Boot problems	System fails to boot or stuck in emergency mode

3 WHAT CHANGED?

- Recent deployments or releases.
- Configuration changes.
- Package updates.
- New users or permissions.
- Infrastructure or network changes.
- Check timestamps in logs and configs.

Most production issues are caused by a change.

4 USEFUL COMMANDS FOR TROUBLESHOOTING

Command	What It Does
`systemctl`	Manage and check service status
`journalctl`	View system and service logs
`ps`	View running processes
`ss`	Check open ports and network connections
`df`	Check disk space usage
`free`	Check memory usage
`ip`	View network configuration
`curl`	Test HTTP/HTTPS connectivity

5 EXAMPLE COMMANDS

```
# Check service status
systemctl status nginx
# View recent logs
journalctl -u nginx -n 50
# Find high CPU processes
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu
# Check open ports
ss -tulnp
# Check disk usage
df -h
# Check memory usage
free -h
# Check network configuration
ip addr show
# Test external connectivity
curl -I https://www.google.com
```

6 TROUBLESHOOTING FLOW

Observe (Identify the symptoms)
↓
Gather Evidence (Check logs, metrics, configs)
↓
Hypothesis (What could be the cause?)
↓
Test (Validate your theory)
↓
Fix (Apply the correct solution)
↓
Verify (Confirm the issue is resolved)

7 RESCUE AND RECOVERY AWARENESS

- Know how to boot into rescue mode.
- Be able to reset the root password.
- Understand how to recover from filesystem or boot issues.
- Keep backups of critical data and configurations.
- Document recovery procedures.

Disaster recovery skills are essential for production engineers.

8 EXAMPLE SCENARIOS

- Website is down → Check service status, logs, and ports.
- High CPU → Identify top processes and investigate.
- Disk full → Find large files and clean up safely.
- Permission denied → Check user, group, and SELinux context.
- DNS failure → Verify /etc/resolv.conf and network connectivity.
- Boot failure → Use rescue mode and check logs.

9 JUNIOR ENGINEER TIP

- Don't guess. Always gather evidence.
- Use logs, metrics and system tools.
- Understand the system before making changes.
- Learn from every incident.
- Build a troubleshooting checklist and keep improving.

Troubleshooting is a skill.
The more you practice, the better you become.

10 KEY TAKEAWAY

OBSERVE. INVESTIGATE. SOLVE. PREVENT. KEEP PRODUCTION RUNNING.

FINAL PRODUCTION LAB

ADMINISTER AND RECOVER A RHEL SERVER

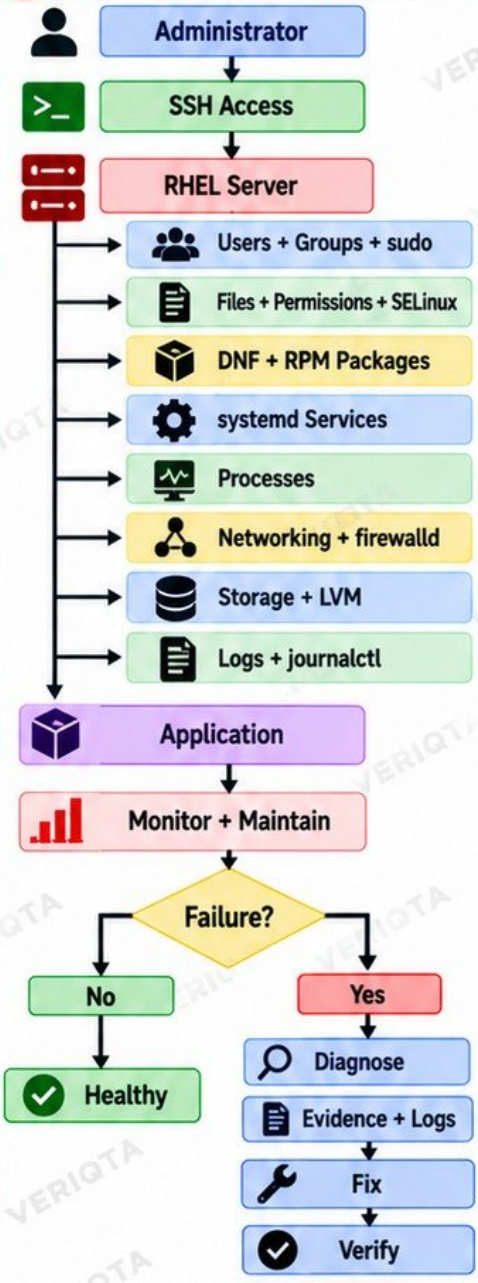
Red Hat
Same Systems Stronger Engineers

Bring It All Together. Real Skills. Real Scenarios. Production Ready.

1 LAB OBJECTIVES (STEP BY STEP)

1. **Connect securely using SSH**
   Use your SSH key to connect to the RHEL server.
2. **Inventory the server**
   Check OS version, hostname, IP, disk, memory and running services.
3. **Create an administrative user**
   Add a new user for administration.
4. **Configure controlled sudo access**
   Give the user sudo access (least privilege).
5. **Install an application package**
   Use dnf to install a useful package (e.g. nginx or httpd).
6. **Configure and start its systemd service**
   Configure the service and start it.
7. **Enable service startup at boot**
   Ensure the service starts automatically at boot.
8. **Configure application files and permissions**
   Set proper file ownership and permissions.
9. **Configure firewalld**
   Allow the required service through the firewall.
10. **Verify listening ports**
    Confirm the application is listening on the expected port.
11. **Inspect networking**
    Check IP, routes, DNS and connectivity.
12. **Create and mount storage**
    Create a partition or LVM, format, mount and verify.
13. **Verify SELinux configuration**
    Check SELinux mode and file contexts.
14. **Inspect application logs**
    Use journalctl and application logs to verify it's running.
15. **Create a simple scheduled administrative task**
    Add a cron job or systemd timer.
16. **Introduce a deliberate configuration failure**
    Break the service (e.g. wrong config) and observe the issue.
17. **Diagnose the failed service**
    Use logs, status and system tools to find the root cause.
18. **Correct the underlying problem**
    Fix the configuration and restart the service.
19. **Verify service recovery**
    Confirm the service is running correctly again.
20. **Perform a final production health check**
    Review system status, services, disk, memory, logs and security.

2 FINAL MENTAL MODEL



This lab brings together everything you have learned. It simulates real administration, troubleshooting and recovery in a production environment.

Practice this lab multiple times until you can complete it confidently from start to finish.

REAL ADMINS SOLVE PROBLEMS. YOU CAN DO THIS.

LEARN PRACTICE OPERATE GROW

3 KEY TAKEAWAY

A well-administered server is secure, monitored, recoverable and ready for production.
Skills + Practice = Confidence.

SAME SYSTEMS. STRONGER ENGINEERS.